CUSTOM VPC SETUP WITH BASTION HOST AND NAT GATEWAY

Services: VPC, EC2, NAT Gateway, Bastion Host, Security Groups

Goal: Build a secure private/public subnet architecture with internet access for private instances.

Skills: Routing tables, subnetting, SSH tunneling.

Definition

This project demonstrates how to build a **secure AWS network architecture** using a **Custom Virtual Private Cloud (VPC)** with **public and private subnets**. A **Bastion Host** is deployed in the public subnet to securely access private instances, while a **NAT Gateway** allows private instances to access the internet for updates without exposing them directly to the public network.

This architecture follows **best practices used in real-world cloud infrastructure** for improving security and controlling network traffic.

Scope of the Project

The scope of this project includes:

- Creating a **custom VPC** with a defined CIDR block.
- Creating **public and private subnets** within the VPC.
- Configuring an **Internet Gateway** for public internet access.
- Creating a **Bastion Host** in the public subnet to access private instances.
- Launching a **Private EC2 instance** that is not directly accessible from the internet.
- Creating a **NAT Gateway** to allow outbound internet access from private instances.
- Configuring **Route Tables and Security Groups** to control traffic flow.
- Demonstrating secure SSH access using **SSH tunneling via Bastion Host**.

Methodologies

The project follows a **secure cloud networking approach** using the following methodology:

1. **Network Segmentation**
    - Separate public and private resources using different subnets.

2. **Controlled Access**
    - Only the Bastion Host has a public IP.
    - Private instances can only be accessed through the Bastion.
3. **Outbound Internet Access**
    - Private instances use a NAT Gateway for internet access without being exposed publicly.
4. **Security Groups**
    - Restrict inbound SSH access to trusted sources.
5. **Routing Control**
    - Public subnet routes traffic through the Internet Gateway.
    - Private subnet routes traffic through the NAT Gateway.

Architecture Diagram / Flow

```
      Internet
         │
  Internet Gateway
         │
┌──────────────────────────┐
│  Public Subnet   │
│            │
│  Bastion Host    │
│    (EC2)       │
└────────────┬────────┘
         │
    SSH Access
         │
┌──────────────────────────┐
│  Private Subnet  │
│            │
│ Private EC2      │
│            │
└─────────────┬────────────┘
         │
    NAT Gateway
```

|
Internet

Explanation of AWS Services Used

1. Amazon VPC

Amazon Virtual Private Cloud provides a logically isolated network where AWS resources can be launched securely.

Purpose:

- Create custom networking environment
- Control IP addressing, routing, and security

---

2. Subnets

Public Subnet

- Allows direct internet access.
- Used to host Bastion Host.

Private Subnet

- Does not allow direct internet access.
- Used to host backend/private servers.

---

3. Internet Gateway (IGW)

An Internet Gateway enables communication between the VPC and the internet.

Purpose:

- Allows resources in the public subnet to access the internet.

---

4. NAT Gateway

A NAT Gateway allows private instances to connect to the internet for updates without exposing them to incoming internet traffic.

Purpose:

- Provide outbound internet access to private instances.

---

5. EC2 Instances

Two EC2 instances were used:

Bastion Host

- Located in Public Subnet
- Acts as a secure jump server

Private Server

- Located in Private Subnet
- No public IP
- Accessible only through Bastion

---

6. Security Groups

Security Groups act as virtual firewalls controlling inbound and outbound traffic.

Bastion Security Group:

- Allows SSH access from user's IP.

Private Instance Security Group:

- Allows SSH only from Bastion Security Group.

---

7. Route Tables

Route tables control how traffic flows within the VPC.

Public Route Table:

0.0.0.0/0 → Internet Gateway

Private Route Table:

0.0.0.0/0 → NAT Gateway

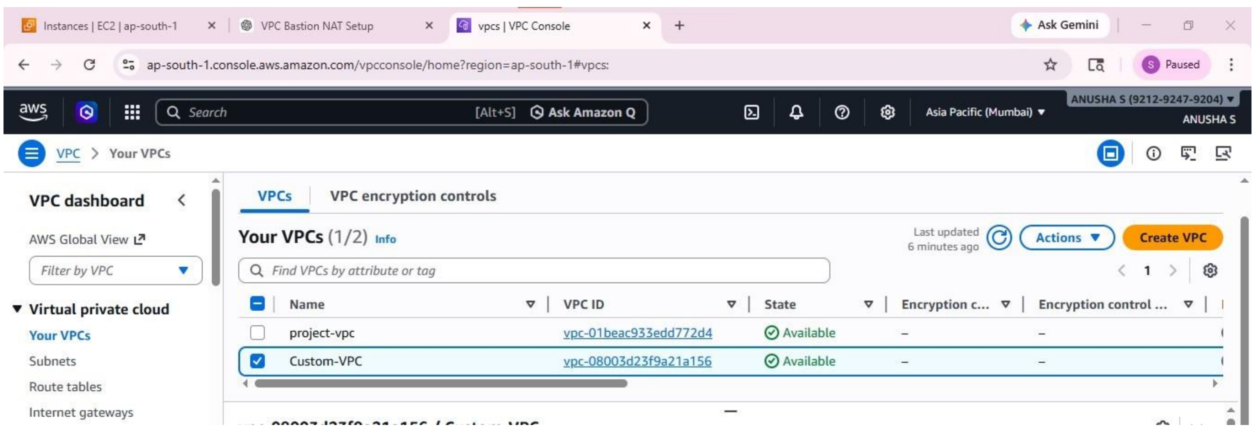
Step-by-Step Procedure

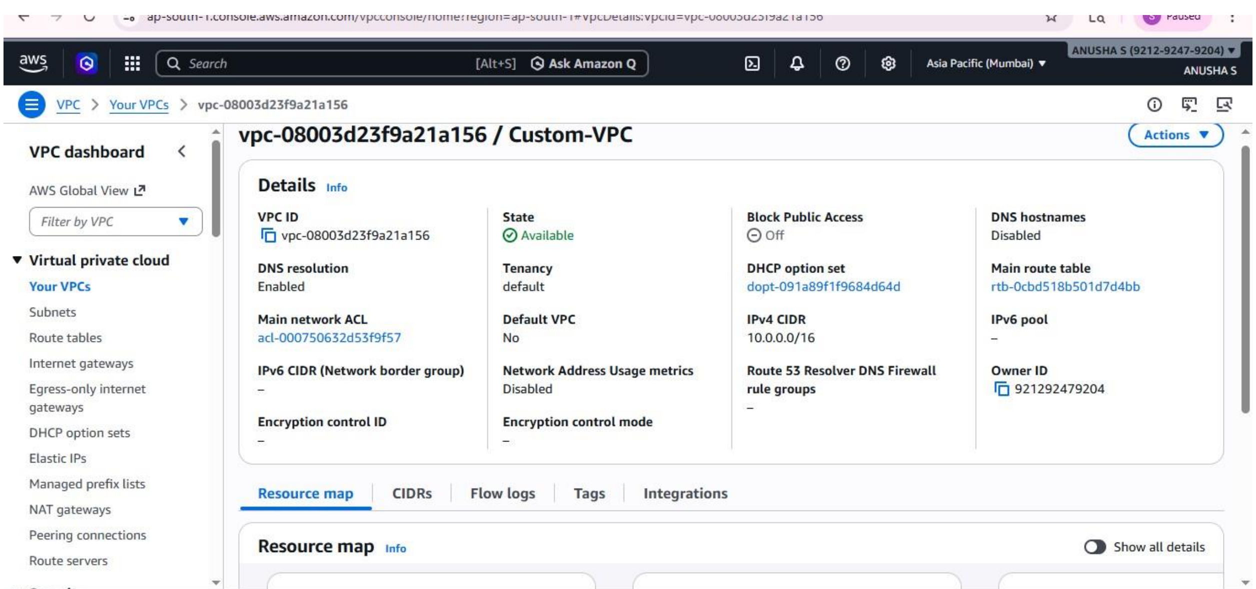
Step 1: Create Custom VPC

- Navigate to VPC Dashboard.
- Create VPC with CIDR block:

10.0.0.0/16

This provides 65,536 private IP addresses.



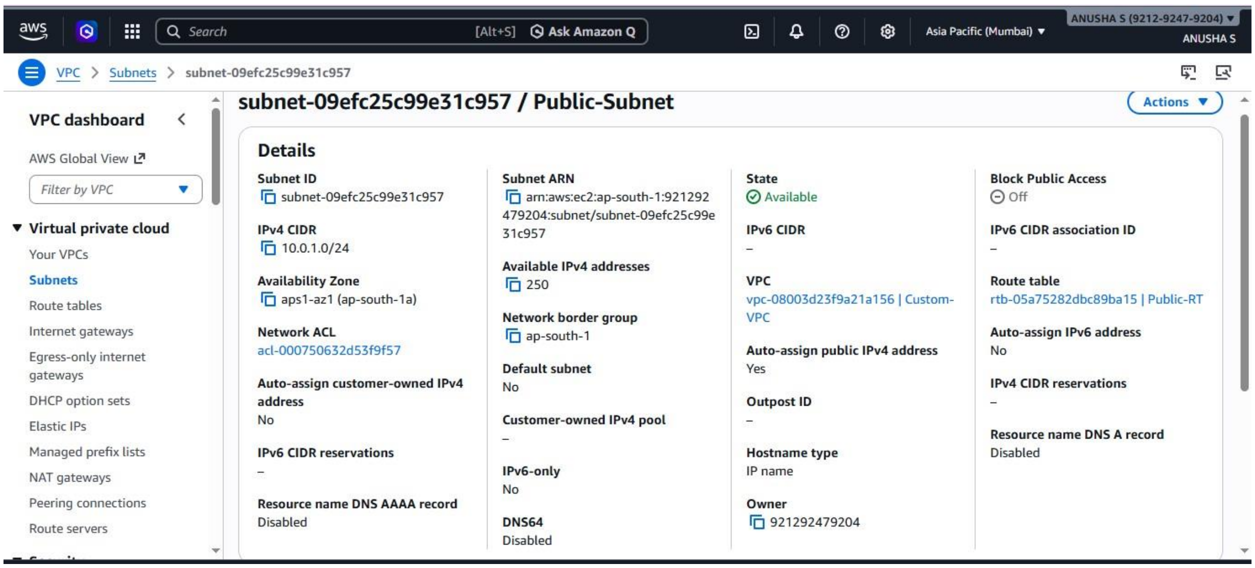


Step 2: Create Subnets

Two subnets were created:

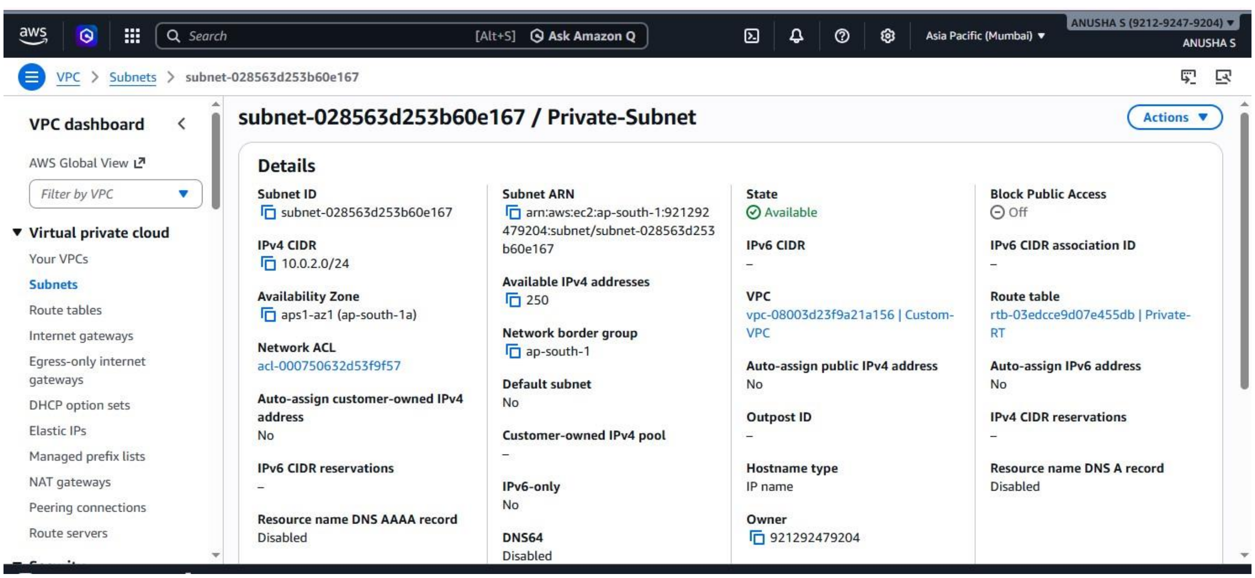
Public Subnet

CIDR: 10.0.1.0/24



Private Subnet-

CIDR: 10.0.2.0/24

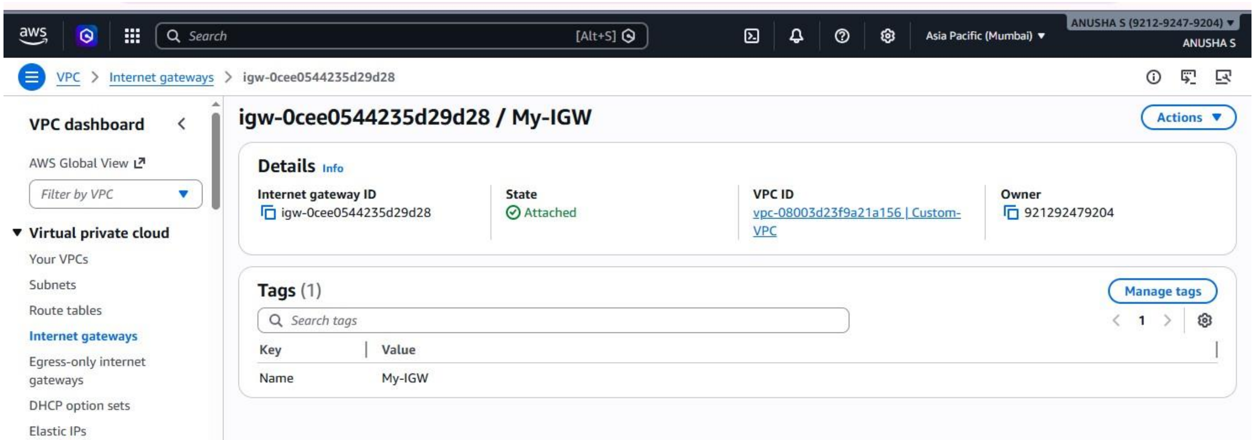


Public subnet was configured to auto assign public IP.

Step 3: Create Internet Gateway

- Create Internet Gateway
- Attach it to the custom VPC.

This enables internet connectivity for the public subnet.

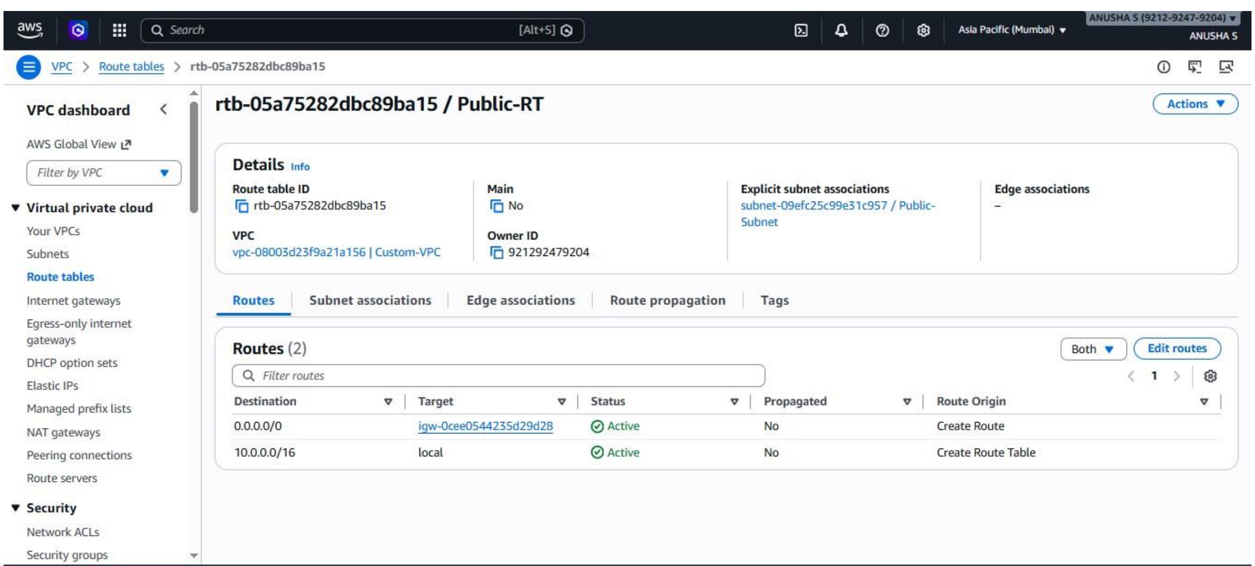


Step 4: Configure Public Route Table

Create a route table for the public subnet and add:

Destination: 0.0.0.0/0
Target: Internet Gateway

Associate the public subnet with this route table.



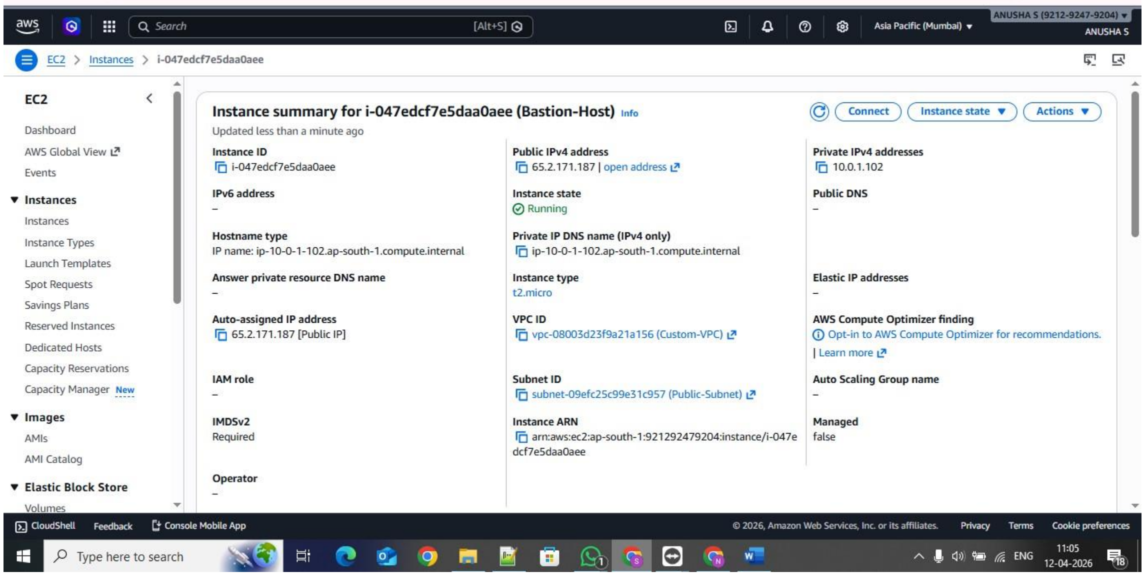
Step 5: Launch Bastion Host

Launch EC2 instance in the public subnet.

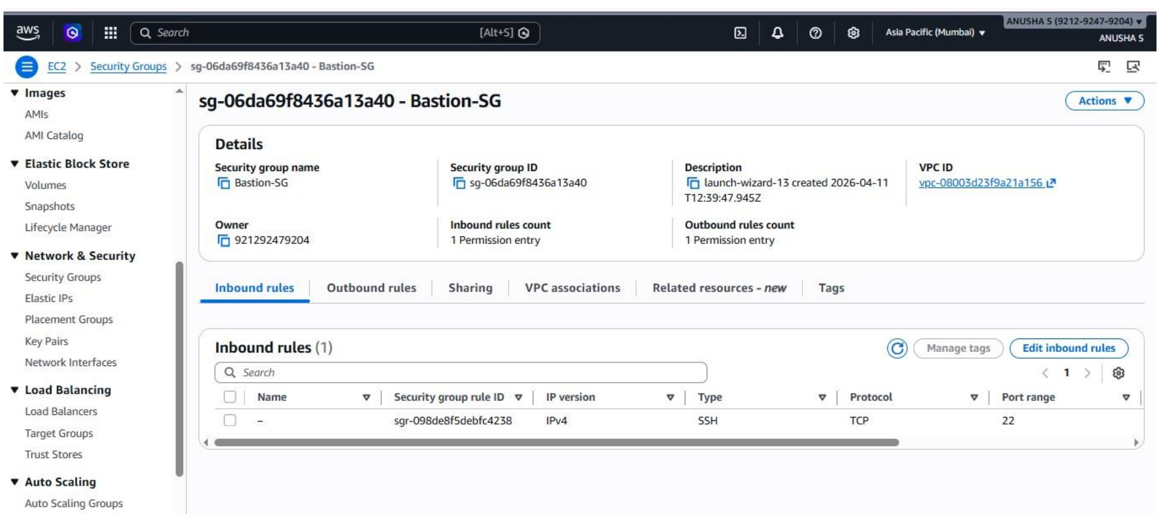
Configuration:

- Instance type: t3.micro

  Public IP enabled



- **Security group allows SSH from user IP.**



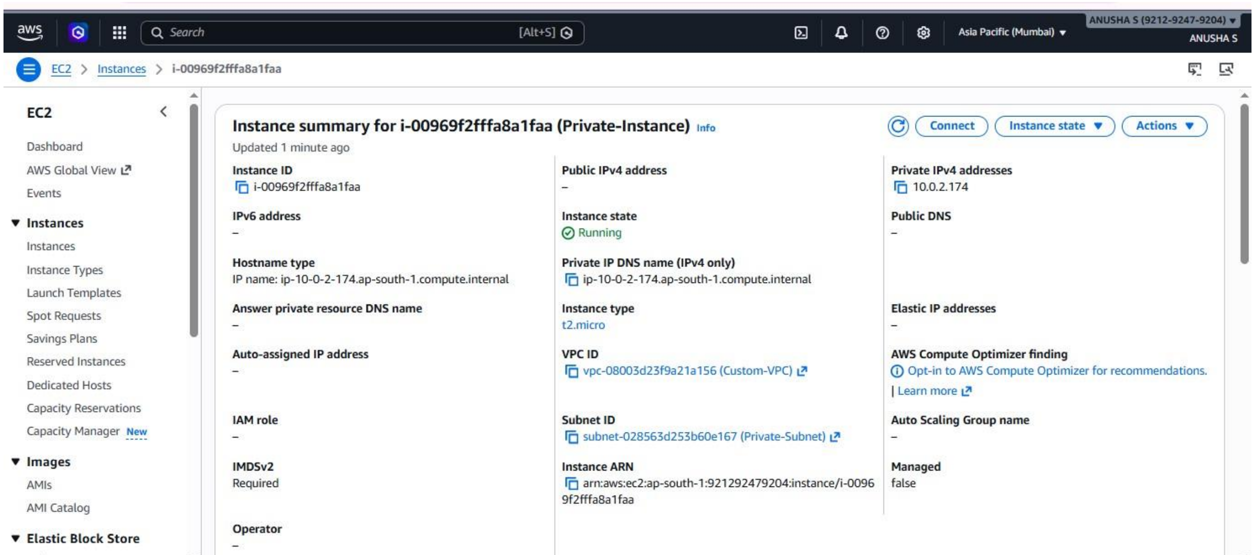
Purpose:
Used as a secure entry point to the private network.

Step 6: Launch Private EC2 Instance

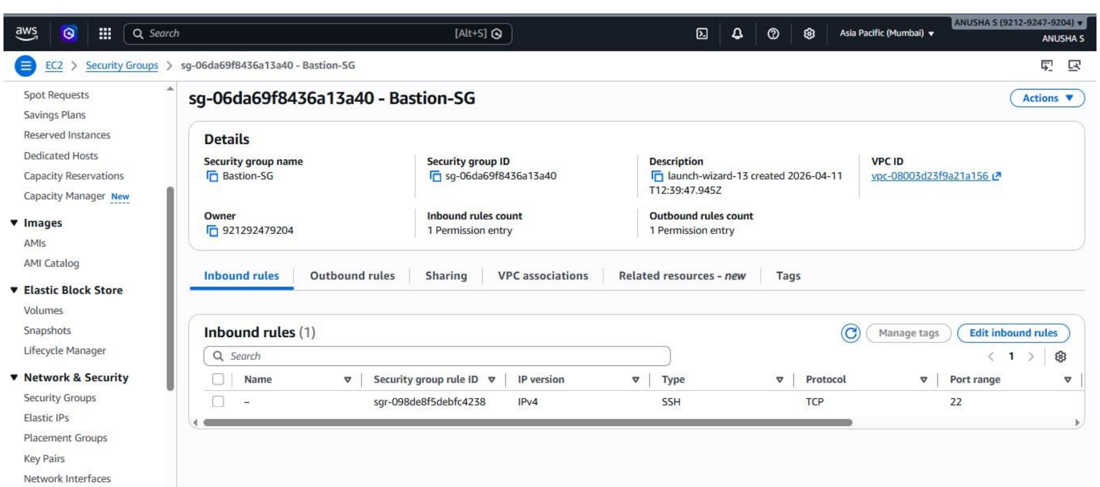
Launch another EC2 instance in the private subnet.

Configuration:

- No public IP
- Private-server
- Private IP only



Security group allows SSH only from Bastion.



Step 7: Connect Using Bastion Host

SSH connection flow:

Laptop → Bastion Host → Private Instance

Commands used:

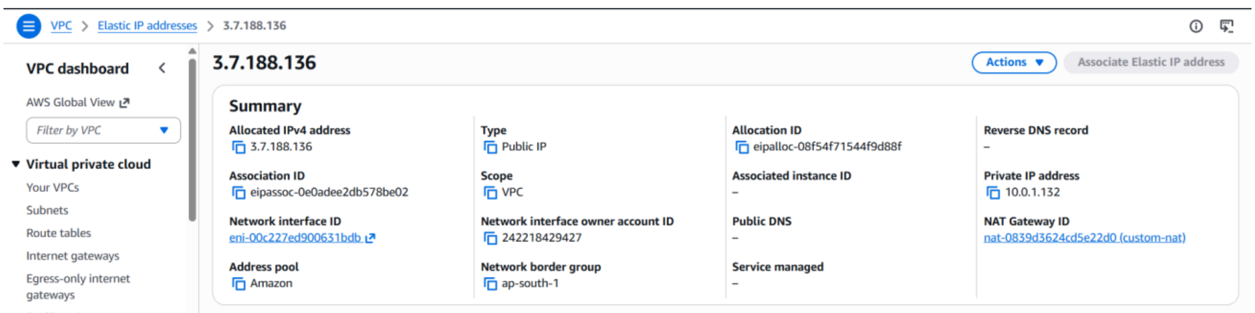
ssh -A -i key.pem ec2-user@Bastion-Public-IP

Then from bastion:

ssh ec2-user@Private-IP

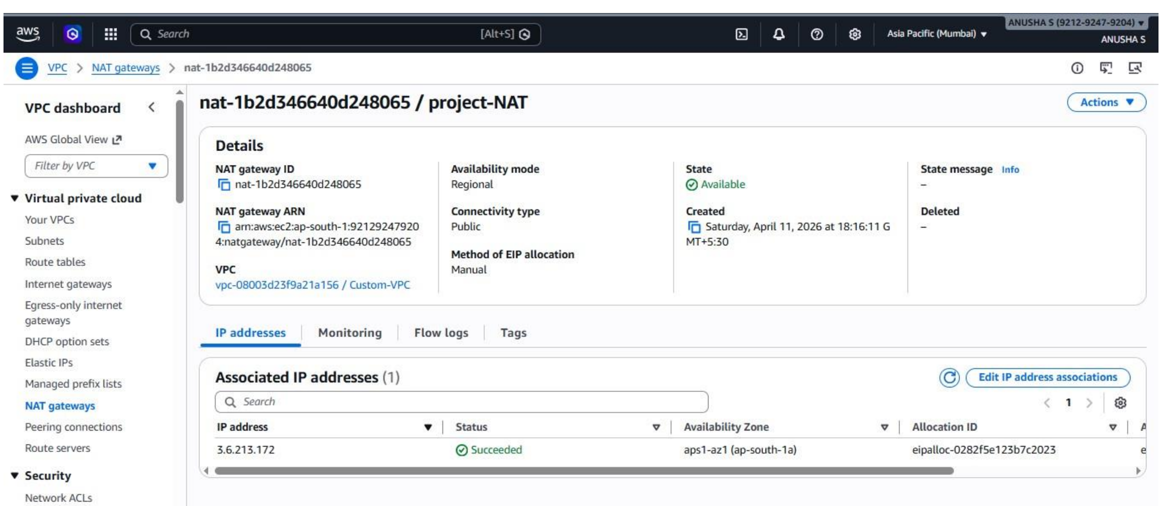
Step 8: Create NAT Gateway

- Allocate Elastic IP



- Create NAT Gateway in Public Subnet.

Purpose:
Allow private instances to access the internet.

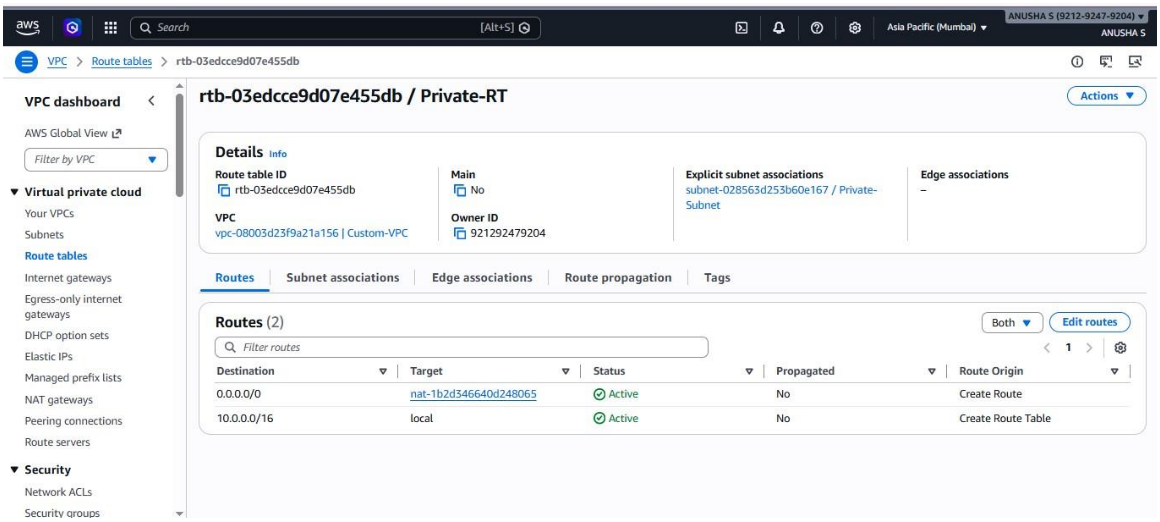


Step 9: Configure Private Route Table

Add route:

Destination: 0.0.0.0/0
Target: NAT Gateway

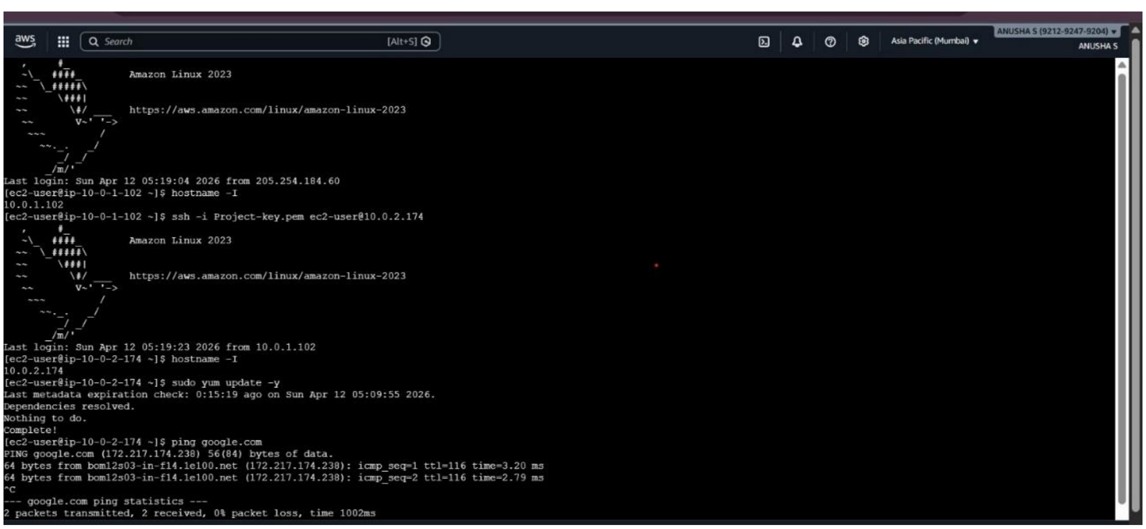
Associate the private subnet with this route table.



Step 10: Test Internet Connectivity

Login to private instance and run:

sudo yum update -y



Successful updates confirm NAT gateway is working

PART 1 — From Laptop (Git Bash)

```
PS C:\Users\Anusha S Reddy> ssh -i .\Project-key.pem ec2-user@65.2.171.187
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Sun Apr 12 05:23:25 2026 from 13.233.177.4
[ec2-user@ip-10-0-1-102 ~]$ |
```

PART 2 — Inside Bastion

Show Private IP of Bastion

```
[ec2-user@ip-10-0-1-102 ~]$ hostname -I
10.0.1.102
```

SSH Into Private Server

```
[ec2-user@ip-10-0-1-102 ~]$ ssh -i ssh -i Project-key.pem ec2-user@10.0.2.174
Warning: Identity file ssh not accessible: No such file or directory.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Sun Apr 12 05:24:49 2026 from 10.0.1.102
```

PART 3 -Inside Private Server

Confirm Private IP

```
[ec2-user@ip-10-0-2-174 ~]$ hostname -I
10.0.2.174
```

It Has No Public IP

Private instance does not have public IP. Outbound traffic goes through NAT Gateway.

```
[ec2-user@ip-10-0-2-174 ~]$ curl ifconfig.me
3.6.213.172[ec2-user@ip-10-0-2-174 ~]$ 
```

Internet Access Works

```
3.6.213.172[ec2-user@ip-10-0-2-174 ~]$ sudo yum update -y
Last metadata expiration check: 1:01:30 ago on Sun Apr 12 05:09:55 2026.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-10-0-2-174 ~]$ ping google.com
PING google.com (142.250.206.174) 56(84) bytes of data.
64 bytes from lcboma-bg-in-f14.1e100.net (142.250.206.174): icmp_seq=1 ttl=116 time=6.73 ms
64 bytes from del11s22-in-f14.1e100.net (142.250.206.174): icmp_seq=2 ttl=116 time=2.57 ms
64 bytes from del11s22-in-f14.1e100.net (142.250.206.174): icmp_seq=3 ttl=116 time=2.32 ms
^C
--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 2.321/3.873/6.733/2.024 ms
```

Troubleshooting

Issue 1

SSH connection timeout.

Method:
Check security group inbound rules.

Solution:
Allow port 22 from user IP.

Issue 2

Unable to connect to private instance.

Method:
Check security group source rule.

Solution:
Allow SSH from Bastion Security Group.

Issue 3

Private instance cannot access internet.

Method:
Verify NAT gateway and route table configuration.

Solution:
Add route:

0.0.0.0/0 → NAT Gateway

Issue 4

Permission denied while SSH.

Method:
Verify key pair used during instance launch.

Solution:
Use correct private key and enable SSH agent forwarding.

Conclusion

This project successfully demonstrates how to design and implement a secure AWS VPC architecture using public and private subnets. By implementing a Bastion Host and NAT Gateway, the system ensures that private resources remain protected while still having controlled access to external services.

This architecture reflects industry best practices for secure cloud networking and is widely used in production environments for hosting web applications and backend services.